Object Oriented Programming with Python

Arslan Arshad (2K12-BSCS-37)

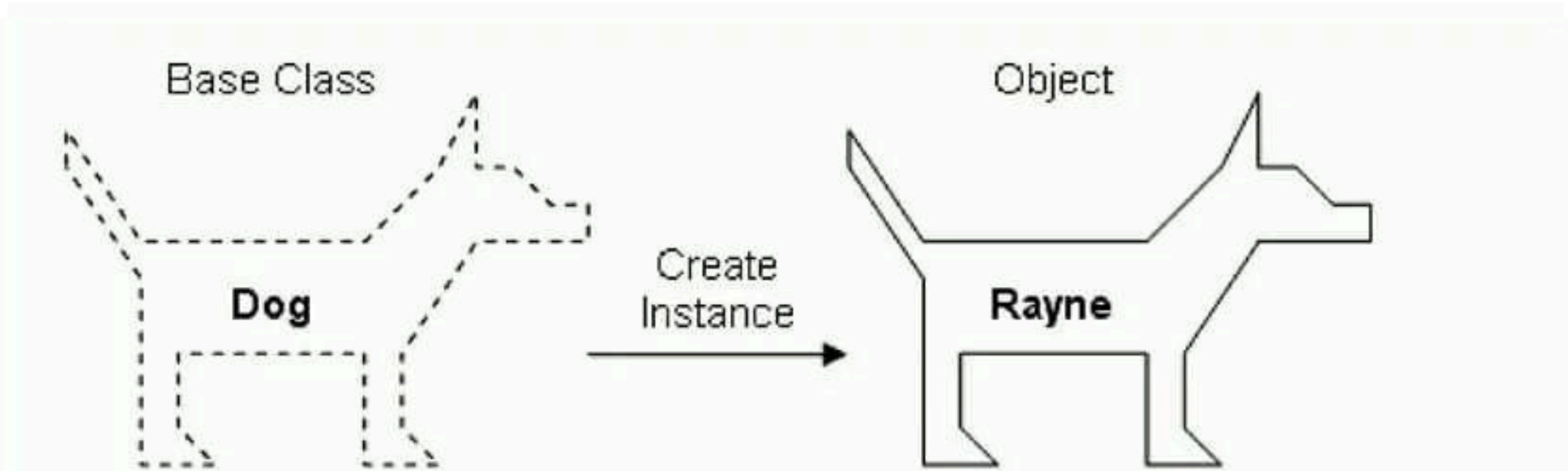
What are Objects?

- An **object** is a location in memory having a value and possibly referenced by an identifier.
- In Python, every piece of data you see or come into contact with is represented by an object.

- Each of these objects has three components:
  I. Identity
  II. Type
  III. Value

```
>>>str = "This is a String"
>>>dir(str)
.....
>>>str.lower()
'this is a string'
>>>str.upper()
'THIS IS A STRING'
```

Defining a Class



Properties	Methods
Color	Sit
Eye Color	Lay Down
Height	Shake
Length	Come
Weight	

Property values	Methods
Color: Gray, White, and Black	Sit
Eye Color: Blue and Brown	Lay Down
Height: 18 Inches	Shake
Length: 36 Inches	Come
Weight: 30 Pounds	

Defining a Class

- Python's class mechanism adds classes with a minimum of new syntax and semantics.
- It is a mixture of the class mechanisms found in C++ and Modula-3.
- As C++, Python class members are public and have Virtual Methods.

- A basic class consists only of the *class* keyword.
- Give a suitable name to class.
- Now You can create class members such as data members and member function.

The simplest form of class definition looks like this:

```
class className:
    <statement 1>
    <statement 2>
    .
    .
    .
    <statement N>
```

Defining a Class

- As we know how to create a Function.
- Create a function in class MyClass named func().
- Save this file with extension .py
- You can create object by invoking class name.
- Python doesn't have new keyword
- Python don't have new keyword because everything in python is an object.

```
class MyClass:
        """A simple Example
of class"""
        i=12
        def func(self):
                return "Hello World
```

```
>>>obj = MyClass(  )
>>> obj.func()
 'Hello World'
```

Defining a Class

```
1.  class Employee:
2.          'Common base class for all employees'
3.          empCount = 0

4.          def __init__(self, name, salary):
5.                  self.name = name
6.                  self.salary = salary
7.                  Employee.empCount += 1

8.          def displayCount(self):
9.                  print("Total Employee %d" % Employee.empCount)

10.         def displayEmployee(self):
11.                 print("Name : ", self.name, ", Salary: ", self.salary)
```

Defining a Class

- The first method *__init__()* is a special method, which is called class constructor or initialization method that Python calls when you create a new instance of this class.
- Other class methods declared as normal functions with the exception that the first argument to each method is self.
- Self: This is a Python convention. There's nothing magic about the word self.
- The first argument in __init__() and other function gets is used to refer to the instance object, and by convention, that argument is called self.

Creating instance objects

```
"This would create first object of Employee class"
emp1 = Employee("Zara", 2000 )
"This would create second object of Employee class"
emp2 = Employee("Manni", 5000)
```

- To create instances of a class, you call the class using class name and pass in whatever arguments its __*init*__ method accepts.
- During creating instance of class. Python adds the *self* argument to the list for you. You don't need to include it when you call the methods

```
emp1.displayEmployee()
emp2.displayEmployee()
print "Total Employee %d" % Employee.empCount
```

Output

```
Name : Zara ,Salary: 2000
Name : Manni ,Salary: 5000
Total Employee 2
```

Special Class Attributes in Python

- Except for self-defined class attributes in Python, class has some special attributes. They are provided by object module.

Attributes Name	Description
__dict__	Dict variable of class name space
__doc__	Document reference string of class
__name__	Class Name
__module__	Module Name consisting of class
__bases__	The tuple including all the superclasses

Form and Object for Class

- Class includes two members: form and object.
- The example in the following can reflect what is the difference between object and form for class.

```
class A:
    i = 123
    def __init__(self):
        self.i = 12345

print A.i
print A().i

>>>
123
12345
```

Invoke form: just invoke data or method in the class, so i=123

Invoke object: instantialize object Firstly, and then invoke data or Methods.
Here it experienced __init__(), i=12345

Class Scope

- Another important aspect of Python classes is scope.
- The scope of a variable is the context in which it's visible to the program.
- Variables that are available everywhere (**Global Variables**)
- Variables that are only available to members of a certain class (**Member variables**).
- Variables that are only available to particular instances of a class (**Instance variables**).